Technical and System Design Document

Four-wheel omnidirectional robot controlled by two-handed gestures
AI-Powered Omni Robot project, version 2

Tai Nguyen Anh

November 2026

Contents

1 System Overview **2**
1.1 Problem and Design Goals . . . 2
1.2 Targets and Measured Results . . . 2
1.3 Overall Architecture . . . 2

2 Mechanics and Kinematics **3**
2.1 Configuration . . . 3
2.2 Inverse Kinematics and Direction Preservation . . . 4
2.3 Forward Kinematics . . . 4

3 Hardware and Electrical Design **4**
3.1 Power and Driver Stage . . . 4
3.2 GPIO Allocation . . . 5

4 Software **6**
4.1 Two-Hand Recognition . . . 6
4.2 Interface Cursor . . . 7
4.3 Communication Protocol . . . 8
4.4 Closed-Loop Control on the ESP32 . . . 8
4.5 Web Interface . . . 9

5 Verification and Data Analysis **10**
5.1 Automatic Tests . . . 10
5.2 Real Run Data . . . 12
5.3 Second Session: Vision Loop with Hands, End-to-End Latency and the Cursor near the Frame Edge . . . 14

6 Limitations and Reproduction **16**

1 System Overview

1.1 Problem and Design Goals

A differential-drive robot cannot slide sideways, whereas an omnidirectional robot with four wheels in a 45° X arrangement moves in any direction on the plane with three independent velocities: forward (v_x), sideways (v_y) and rotation (ω). Gesture interfaces usually handle only driving, while changing mode, setting speed or pressing the emergency stop still needs a mouse or keyboard.

The project does both with a single laptop camera:

- The **dominant hand** controls the web interface: it moves the cursor, holds still to click, and raises the middle finger to press or drag the virtual joystick and the sliders.
- The **non-dominant hand** drives the robot with 8 pointing directions and a V sign for rotation. In Manual mode (virtual joystick, keyboard) any single hand is enough to operate the interface.

The robot hardware is an ESP32 with two L298N H-bridges and four GM25-370 motors with encoders. The ESP32 creates its own Wi-Fi network and receives commands over UDP.

1.2 Targets and Measured Results

The targets in Table 1 were set by the team as reference marks, not as external requirements. The results column comes from the measurements in Section 5.

Table 1: Targets and measured results.

Item	Target	Measured result
Camera + hand detection loop rate	≥ 25 fps	29.9 fps without hands (isolated benchmark); 18.0–25.5 fps in server sessions, inference 45.5–55.5 ms with one hand
UDP round-trip time (RTT)	< 50 ms (median)	median 6.6–8.1 ms; P95 16.1–26.1 ms
Per-wheel speed tracking error (mean)	$< 5.0\,\%$	1.5–3.2 % of full speed; P95 4.4–7.4 %
Body velocity deviation from command	$< 5.0\,\%$	0.1–2.5 % on the main axis; other axes ≤ 0.004
From confirmed gesture to wheel at 50		
,		
Fail-safe stop on lost command	< 500 ms	ESP32 300 ms; laptop 400 ms; jam protection 700 ms

Note: the 29.9 fps and 18.5 ms inference were measured on 12 November 2026 over 359 consecutive frames; the gesture-to-wheel latency includes the deliberate acceleration limit (Section 4.4) and excludes the 4 frames (about 133 ms) needed to confirm a gesture.

1.3 Overall Architecture

The system has two parts connected by a Wi-Fi network created by the ESP32 itself (SSID `omni-robot`, fixed IP `192.168.4.1`, UDP port 4210), with no Internet needed (Fig. 1).

- **Laptop (host)**: `ai/camera.py` reads MJPG 1280 × 720 images at 30 fps in a separate thread (detection runs on a 640-px-wide copy) that keeps only the newest frame. MediaPipe HandLandmarker returns 21 landmarks for up to 2 hands. `RoleAssigner` and `split_roles()` send the dominant hand to the cursor controller (`ai/ui/cursor.py`) and the non-dominant hand to the driving gesture classifier (`ai/drive/`). The 20 Hz `control_loop()` selects the command according to the mode, limits its acceleration, and sends it by UDP through `RobotLink`.
- **ESP32 (edge)**: receives the command packet, checks the single-owner lock and the 300 ms timeout, computes inverse kinematics, runs 4 speed controllers every 10 ms, outputs PWM to the two L298N modules, and returns an acknowledgement with the target speed, measured speed and PWM of the 4 wheels.

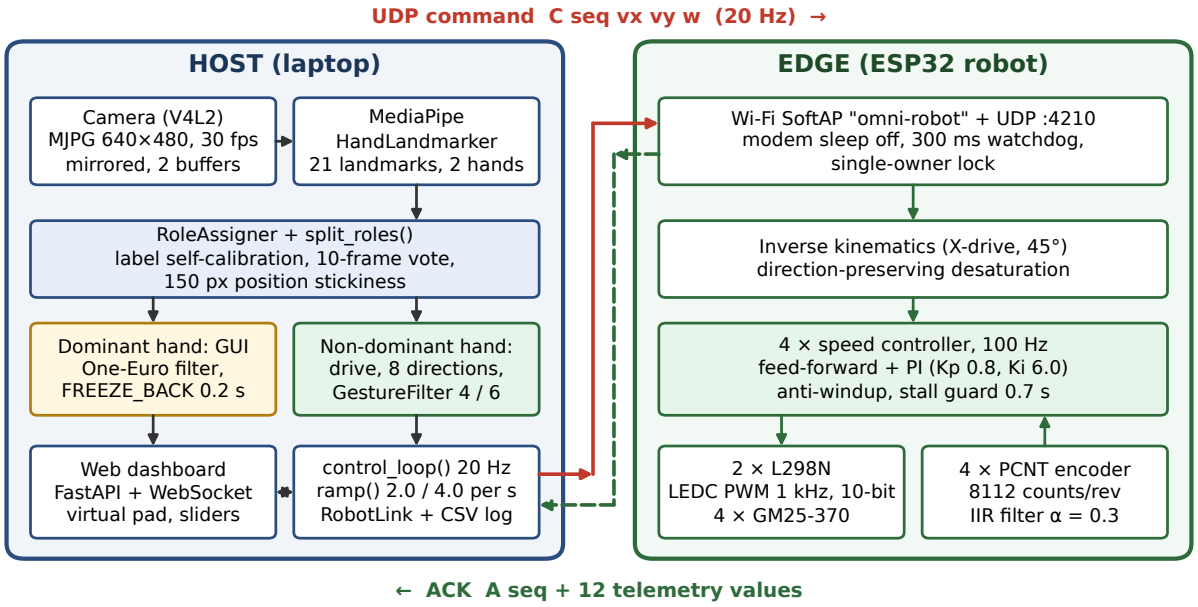


Figure 1: Host–edge architecture. Solid red: UDP command; dashed green: acknowledgement with wheel telemetry.

2 Mechanics and Kinematics

2.1 Configuration

The four omni wheels sit at the four corners with their main rolling axis tilted ±45° from the body's longitudinal axis (X arrangement), directly on the output shafts of four 1:78 geared motors. The code uses the ROS convention: $+x$ forward, $+y$ to the left, and $+\omega$ counter-clockwise seen from above (Fig. 2). Wheel indices: 0 front-left (tt), 1 front-right (tp), 2 rear-left (st), 3 rear-right (sp). Positive wheel speed is the direction that moves the robot forward.

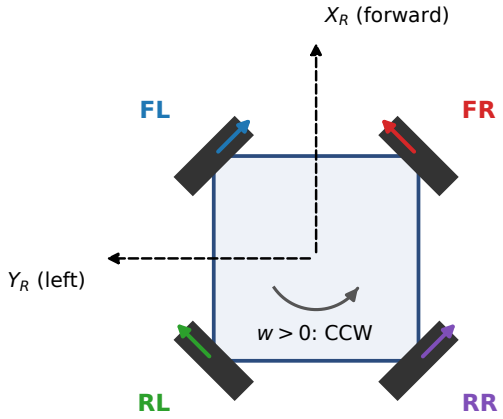


Figure 2: X-configuration geometry and the thrust direction of each wheel when rolling forward.

2.2 Inverse Kinematics and Direction Preservation

In the firmware (`firmware/src/kinematics.h`) velocities are normalized to $[-1, 1]$, with 1.0 corresponding to `MAX_CPS` $= 4300$ counts/s (about 31.8 rpm). With $\mathbf{u} = [v_x, v_y, \omega]^\top$:

$$\begin{bmatrix} r'_{tt} \\ r'_{tp} \\ r'_{st} \\ r'_{sp} \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix} \mathbf{u}, \qquad r_i = \frac{r'_i}{m}, \quad m = \max\left(1, \max_j |r'_j|\right). \tag{1}$$

Dividing by the common factor m keeps the ratio between the wheels when one exceeds 1.0, so the robot only slows down and does not change direction. Clipping each wheel separately would distort the ratio and make the robot drift.

2.3 Forward Kinematics

The columns of the matrix in (1) are orthogonal with squared norm 4, so its pseudo-inverse is the transpose divided by 4. From the measured speeds m_i of the 4 wheels, `tests/analyze_drive_log.py` recomputes the body velocity:

$$\begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}_{\text{odo}} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} m_{tt} \\ m_{tp} \\ m_{st} \\ m_{sp} \end{bmatrix}. \tag{2}$$

3 Hardware and Electrical Design

3.1 Power and Driver Stage

A 12 V main supply feeds the two L298N modules and the four motors. The L298N uses bipolar transistors and therefore has a large internal voltage drop, about 1.5–2 V according to the general characteristics of this IC (not measured separately on this board), which is why 12 V was chosen. The measured maximum speed of the four wheels is 4552–4869 counts/s (33.7–36.0 rpm); `MAX_CPS` is set to 4300, below the slowest wheel, so that the controller has spare PWM

to compensate. PWM uses the ESP32 LEDC peripheral at 1 kHz, 10 bit, because the L298N switches slowly.

The encoders are powered at 3.3 V to match the ESP32 logic level; their outputs are push-pull, so the internal pull-ups are disabled in software, and the four encoders share a common GND wire with the ESP32.

3.2 GPIO Allocation

There are 20 signal pins: 4 PWM, 8 direction pins (IN1, IN2) and 8 encoder pins (phases A and B). The input-only pins of the ESP32 (GPIO 34, 35, 36, 39) are used for the encoders of the two front wheels, leaving output-capable pins for the L298N (Table 2, wiring diagram in Fig. 3).

Table 2: GPIO pins (`firmware/src/robot_config.h`).

Wheel	PWM	IN1	IN2	Encoder A	Encoder B
0 front-left (tt)	25	26	27	36	39
1 front-right (tp)	13	23	22	34	35
2 rear-left (st)	21	19	18	32	33
3 rear-right (sp)	17	16	4	14	5

The two right-hand motors are mounted as mirror images, so their encoders count with the opposite sign: `ENCODER_INVERT` = {false, true, false, true}. The rear-right motor is wired in reverse and compensated by `MOTOR_INVERT` = {false, false, false, true}. As a result, everywhere else in the code a positive PWM always turns the wheel forward and the measured speed is always positive when moving forward.

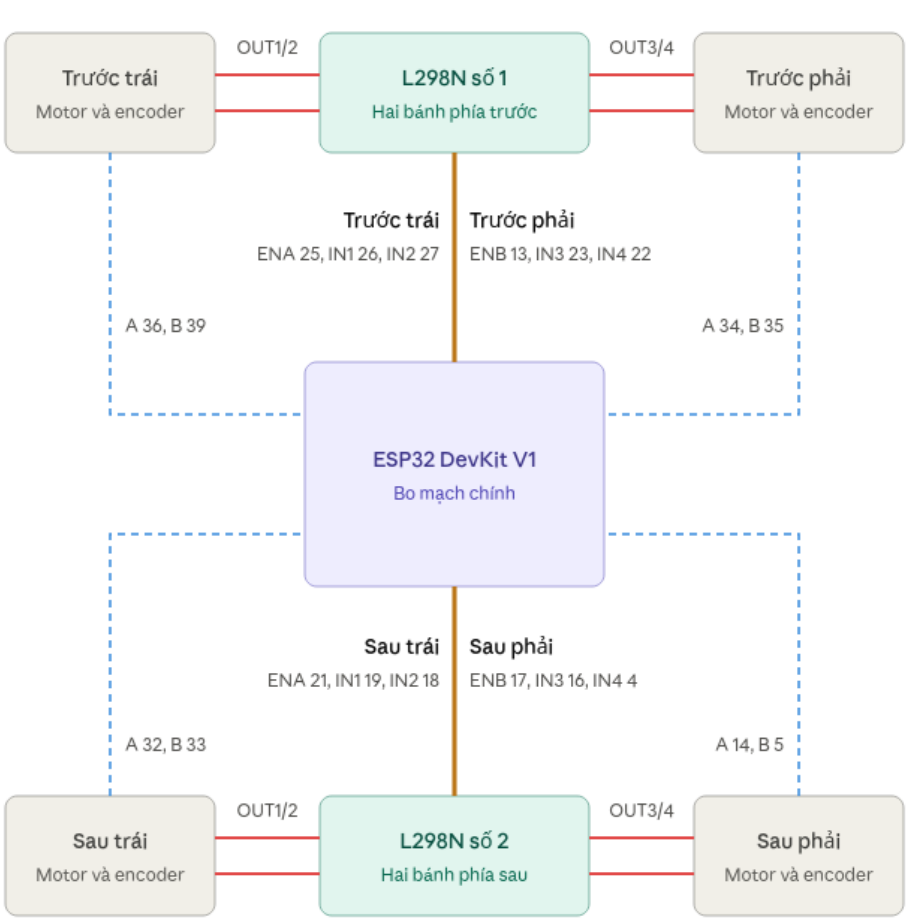


Figure 3: Wiring of the ESP32 DevKit V1, the two L298N modules and the four GM25-370 motors with encoders (`docs/pinout.png`).

4 Software

4.1 Two-Hand Recognition

Recognition uses geometry on the 21 MediaPipe landmarks (VIDEO mode, confidence threshold 0.6), with no additional training.

Extended fingers. A finger is extended when the distance from its tip to the wrist is greater than 1.15 times the distance from its middle joint (PIP) to the wrist. The ratio is dimensionless, so it does not depend on the distance to the camera or on the rotation of the hand. If only the index finger is extended, the pointing direction is computed from the base of the finger (landmark 5) to its tip (landmark 8) and divided into 8 sectors of 45°.

Role assignment. The image is mirrored, but MediaPipe assigns the labels Left/Right as if it were not, so the labels are inverted from the user's point of view. `RoleAssigner` handles this as follows:

1. Two hands: roles follow the horizontal order of the wrists; the disagreement between this order and the (flipped) labels updates a score $c_k = 0.95\, c_{k-1} \pm 0.05$, and when $c_k < -0.6$ the label convention is flipped.

2. One hand: the label is voted over the last 10 frames.

3. Just went from two hands to one and the wrist moved less than 150 px: keep the previous role.

4. Manual mode (`drive_active = False`): the driving branch is locked and any hand is the interface hand.

Driving-gesture filter. `GestureFilter` needs 4 consecutive frames (about 0.13 s) to start a motion command and 6 frames (about 0.2 s) to stop, so a single noisy frame does not make the robot jerk.

4.2 Interface Cursor

The dominant hand has three poses: *point* (index finger) moves the cursor, *click* (index plus middle) presses or drags, and *idle* (anything else) hides the cursor to avoid accidental presses. A pose must be held for 3 frames to be accepted; if the hand is lost for more than 0.3 s the cursor is hidden and every held button is released.

The index fingertip position is smoothed with a One-Euro filter ($f_{\min} = 1$ Hz, $\beta = 8$ in normalized coordinates); the cut-off $f_c = f_{\min} + \beta|\dot{x}|$ is low when the hand is still and increases with hand speed.

Action-point anchor (`FREEZE_BACK`). Raising the middle finger to click can pull the index fingertip away from the button being aimed at (measured on real hands in Section 5). When the pose changes from point to click at time t, the system takes the latest filtered position that is at least 0.2 s old as the origin and keeps the compensating offset for the whole click:

$$(x_a, y_a) = (x_j, y_j),\ j = \arg\max\{t_j :\ t - t_j \geq 0.2\}, \qquad (a_x, a_y) = \mathrm{clamp}\big(x + x_a - x(t),\ y + y_a - y(t)\big). \tag{3}$$

The displayed cursor still follows the fingertip, so dragging stays smooth. After release the offset decays exponentially with a time constant of 0.15 s (`OFFSET_TAU`). Sliders use relative grabbing: hold still to grab, move the finger to drag, hold still again to release. Fig. 4 illustrates this on a simulated hand.

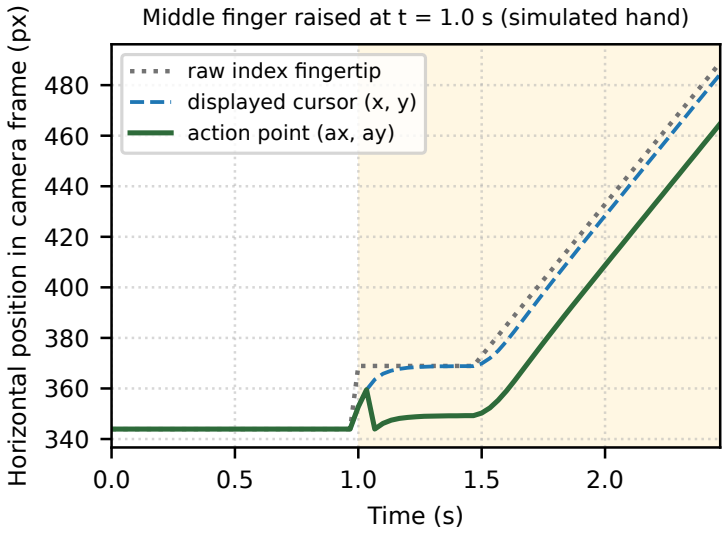


Figure 4: Action-point anchor on a simulated hand (using the actual `Cursor` class). Raising the middle finger at $t = 1.0$ s shifts the fingertip by 25 px.

Mapping near the frame edge. Hand detection fails when the hand is cut by the frame border, so a cursor that coincides with the fingertip cannot reach the corners of the screen. Let

$d = f - \frac{1}{2}$ be the fingertip position in the frame ($f \in [0,1]$, per axis), $a = 0.10$ the half width of the central zone and $b = 0.44$ the distance at which the cursor should reach the screen edge. With $k = (0.5 - b)/(b - a)^2$ the stretched coordinate is $s = \frac{1}{2} + \mathrm{sgn}(d)\min\{\frac{1}{2},\ |d| + k(|d| - a)^2\}$ for $|d| > a$ and $s = f$ otherwise, which keeps the cursor on the fingertip in the middle, grows the lead smoothly without a kink (peak gain 1.35) and reaches the screen edge when the fingertip is 6 % of the frame from the border. If the hand is lost while the fingertip is within 12 % of the border, the cursor and pose are held for 1.0 s instead of 0.3 s. A first version, with a linear ramp starting at $|d| = 0.30$ and peak gain 1.67, made the cursor visibly run ahead of the fingertip and was replaced by this one. Nine of the 55 automatic checks cover this mapping.

4.3 Communication Protocol

For continuous 20 Hz motion commands only the latest value is useful, so UDP was chosen: a dropped packet is ignored and replaced 50 ms later, whereas TCP would hold back newer packets behind a lost one. On the ESP32 the Wi-Fi modem sleep is disabled (`WiFi.setSleep(false)`) to reduce latency. The frame is ASCII text, easy to debug:

`C <seq> <vx> <vy> <w>` (laptop → ESP32), e.g. `C 142 0.500 -0.200 0.000`
`A <seq> <m0 d0 p0 ... m3 d3 p3>` (ESP32 → laptop; target, measured, PWM of the 4 wheels)

Layered safety. (1) The ESP32 accepts commands only from the first valid sender and rejects other addresses until that sender has been silent for more than 300 ms. (2) The ESP32 returns the command $(0, 0, 0)$ if no new packet arrives for 300 ms. (3) The laptop sets the target velocity to 0 if no new command arrives from the selected mode for 0.4 s. (4) The `ramp()` limiter has a maximum acceleration of 2.0/s and a maximum deceleration of 4.0/s, and forces the velocity through 0 before it changes sign.

4.4 Closed-Loop Control on the ESP32

Every 10 ms (100 Hz), `firmware/src/wheel.h` reads the counts from the hardware PCNT counter in ×4 quadrature decoding mode (8112 counts/wheel revolution = 26 counts/channel ×4 × 78) with a 12.5 μs hardware glitch filter, so counting does not use CPU time. Because a single count in 10 ms already equals 2.3 % of full speed, the measured speed is low-pass filtered with a first-order filter:

$$\omega_i(k) = 0.7\,\omega_i(k-1) + 0.3\,\frac{c_i(k)}{4300}, \tag{4}$$

with c_i the counts per second. The target r_i of each wheel moves toward the commanded value with a maximum slope of 4.0/s. The control law (Fig. 5) is:

$$u_i^{\mathrm{ff}} = \mathrm{sgn}(r_i)\big(a_i + b_i|r_i|\big), \quad e_i = r_i - \omega_i, \tag{5}$$

$$I_i \leftarrow I_i + K_i e_i\,\Delta t \quad \text{only when not ramping and not saturated}, \tag{6}$$

$$u_i = \mathrm{clamp}\big(u_i^{\mathrm{ff}} + K_p e_i + I_i,\ -1,\ 1\big), \qquad K_p = 0.8,\ K_i = 6.0. \tag{7}$$

The L298N plus GM25-370 system is strongly non-linear: the PWM threshold at which a wheel starts to turn differs between wheels (5.5 % to 14.8 %, a factor of 2.7) and about 44 % PWM already gives 80 % of the no-load speed. The coefficients a_i, b_i (Table 3) are least-squares lines fitted to steady-state points measured with the wheels lifted, using `firmware/tools/motor_test.cpp`; the maximum fitting deviation is 0.045 PWM. The feed-forward term supplies approximately the right PWM immediately, and the PI term only compensates the remainder (especially the real load on the floor). Freezing the integral while ramping or saturated avoids accumulating error while the target is still moving (windup). There

is no comparison with a plain PID or open loop on the same trajectories, so the improvement has not been quantified.

Jam protection. If the PWM is at least 0.95 while the measured speed stays below 0.05 for more than 0.7 s (`STALL_SECONDS`), that wheel is switched off and an error is reported over Serial until the command returns to 0.

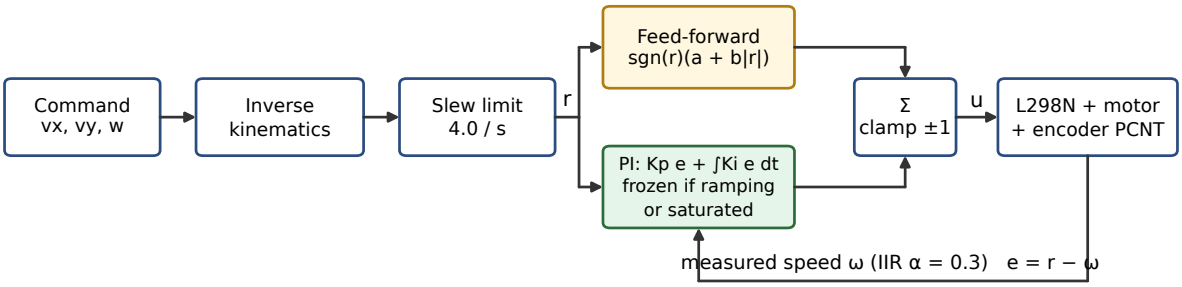


Figure 5: Speed controller of one wheel (100 Hz).

Table 3: Measured feed-forward parameters and mean tracking error on the floor (% of full speed).

Wheel	a_i (`FF_OFFSET`)	b_i (`FF_SLOPE`)	Manual v1	Gesture v1	Two-hand v2
tt	0.076	0.470	1.8	1.9	2.5
tp	0.148	0.594	3.2	2.5	2.6
st	0.055	0.453	1.7	1.6	2.3
sp	0.060	0.464	1.5	1.7	2.5

4.5 Web Interface

The dashboard (FastAPI, WebSocket, single page with no scrolling) has two modes: Manual (virtual joystick, rotation buttons, arrow keys or W A S D, Q E, Space) and Hand Gesture. Every button, slider and joystick can be operated with the dominant-hand cursor; the gesture mapping is in Table 4. Fig. 6 was captured from the running application with no camera attached.

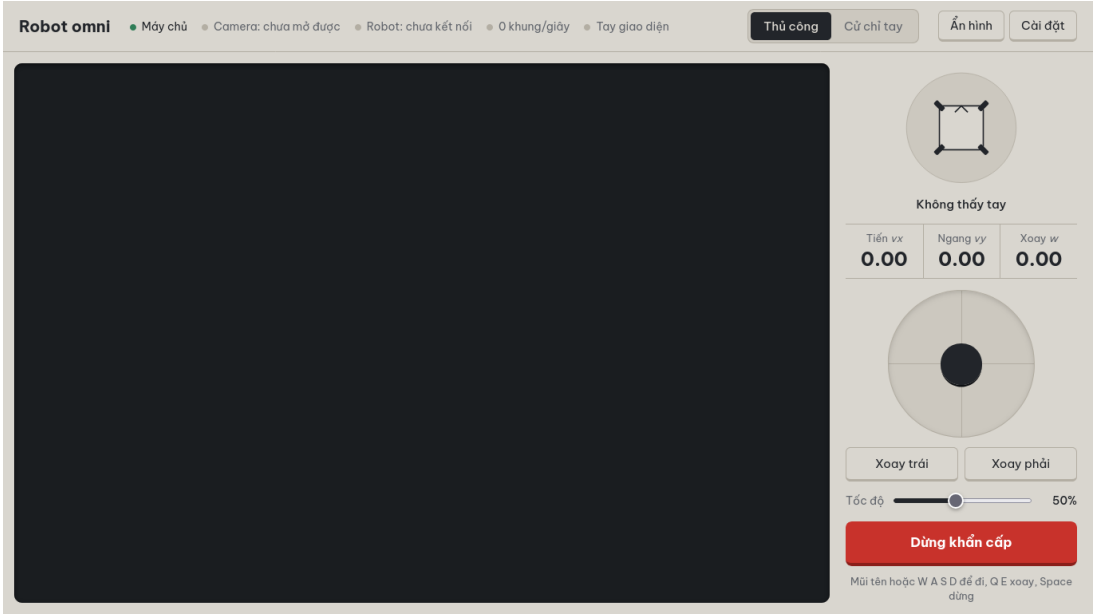


Figure 6: The dashboard in Manual mode.

Table 4: Gesture mapping ($s \in [0.1, 1]$ is the speed factor).

Hand	Gesture	Effect
Non-dominant	fist, open palm, hand lost	Stop: $(0, 0, 0)$
Non-dominant	point up / down	Forward $(+s, 0, 0)$ / backward $(-s, 0, 0)$
Non-dominant	point left / right	Left $(0, +s, 0)$ / right $(0, -s, 0)$
Non-dominant	point diagonally	Diagonal $(\pm 0.707s, \pm 0.707s, 0)$
Non-dominant	V sign left / right	Rotate left $(0, 0, +s)$ / right $(0, 0, -s)$
Dominant	point	Move the cursor; hold still 0.8 s to click or grab a slider
Dominant	point plus middle	Quick click; hold and move to drag
Dominant	other, hand lost	Hide the cursor, release every held button

5 Verification and Data Analysis

5.1 Automatic Tests

- `tests/test_hands_logic.py`: 55 checks on synthetic hands (including the edge mapping of the cursor) built by mathematics, covering gesture classification, interface pose, role assignment with inverted labels, self-calibration, the One-Euro filter and the action-point anchor. Result: 55/55 pass.
- `tests/handcursor_test.html`: 49 checks in a browser for dwell click, two-finger click, relative slider grab, joystick drag, hold buttons, automatic release on hand loss and the info card. Run with headless Chrome (1280×900 window) on 12 November 2026: 49/49 pass. A window that is too short (below about 500 px) makes one cursor-position check fail falsely because the cursor is clamped to the window edge.

The automatic checks use a simulated hand, so their numbers only describe the algorithm on ideal data. With 2 px noise (200 seeded runs) the standard deviation of the cursor fell from 0.0047 to 0.0016 (65.5 % reduction). With the index fingertip displaced by $(25, -15)$ px when the middle finger is raised, the action point shifted by 0.0126 (normalized units) against 0.0770 for the fingertip, a reduction of 83.6 %; the residual is 4.6 px out of 29.2 px. Because the click pose is accepted only after 3 frames, the action point can briefly deviate by up to 15.6 px (about 0.1 s) before compensation.

Measurement on a real hand. We ran `tests/real_hand_eval.py` (the same `Cursor` class, real camera images, one test subject); data in `docs/data/v2/real_hand_click_still.csv` and `real_hand_jitter.json`. *Hold-still jitter:* with the index finger held still for 8 seconds (209 frames), the standard deviation of the raw fingertip was 2.26 px and that of the filtered cursor 2.14 px, a reduction of only 5 % (65.5 % in simulation). The simulation used independent noise, whereas real tremor and landmark drift are probably slow and correlated, so a low-pass filter tuned for responsiveness does not remove them; we have not tested this hypothesis. *Click shift:* the subject pointed at a spot and held still (the script waited until the fingertip stayed within 3 px for 1 second and rejected trials where the hand was still drifting), then raised the middle finger, 12 times. The index fingertip moved by a median of 1.65 px (1.1–5.7), far less than the 29.2 px assumed in the simulation. The anchored action point moved by a median of 4.15 px (2.0–12.0), more than the fingertip in 11 of 12 trials (Fig. 7); the displayed cursor moved 1.9 px. So for this subject, with the finger held still, the 0.2 s look-back added about 2.5 px instead of compensating a shift. A possible reason is that the real shift is of the same size as the hand tremor, so the anchor samples the tremor instead of correcting a systematic jump. An earlier run in which the subject did not keep the finger still gave a median fingertip shift of 6.8 px and is not used. These are 12 trials from one person; the click hit rate was not measured, and the dwell radius (30 px) and the button sizes are larger than these deviations, so the practical effect on pressing is probably small but unverified. For other users or hand shapes with a stronger fingertip jump, the anchor may still help.

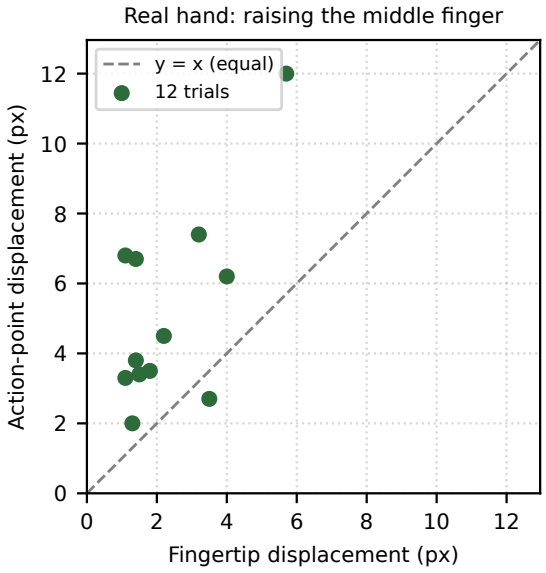


Figure 7: Real hand, 12 trials with the finger held still: displacement of the action point against that of the raw fingertip when the middle finger is raised. Points above the dashed line mean the action point moved more than the fingertip.

5.2 Real Run Data

Three floor runs, logged at 20 Hz, were analyzed with `tests/analyze_drive_log.py`; the data are in `docs/data/v1/` and `docs/data/v2/` (Table 5). Rows recorded while the motor supply was off (53 rows in the manual run) are excluded.

Table 5: The logged runs.

Run (file)	Rows	Duration (s)	RTT median / P95 / max (ms)
Manual v1 (`drive_manual.csv`)	3240	1512	7.8 / 22.5 / 97
Gesture v1 (`drive_gesture.csv`)	4820	247	8.1 / 26.1 / 101
Two-hand v2 (`drive_twohand.csv`)	5020	266	6.6 / 16.1 / 909
Total	13080	2026	–

The manual run contains 389 rows in gesture mode. The maximum RTT in the v2 run was 909 ms, far above the 300 ms limit of the ESP32, so the robot would stop in such a case.

Latency. From the moment a motion gesture is recorded until the most loaded wheel reaches 50 % of its new target: median 260 ms (204–359 ms, 38 events, v1) and 257 ms (203–309 ms, 26 events, v2). These figures include the deliberate acceleration limit and are therefore not the smallest achievable latency.

Gesture stability. Among 120 gesture changes (v1), 7 segments lasted less than 0.3 s (5.8 %); among 76 (v2), 5 segments (6.6 %). We did not annotate whether these short segments were unintended actions.

Speed tracking. Only counted when the command was constant for 0.3 s and at least 0.1 in magnitude. The mean error is 1.5–3.2 % (Table 3, Fig. 8). The 95th percentile is 4.4–4.8 % in the v1 gesture run but 6.1–7.4 % in the v2 two-hand run, so the 5 % target is met by the mean but not by the 95th percentile in the v2 run. The mean PWM of the tp wheel is the highest (0.48–0.49 against 0.36–0.44 for the other wheels), consistent with its largest dead zone.

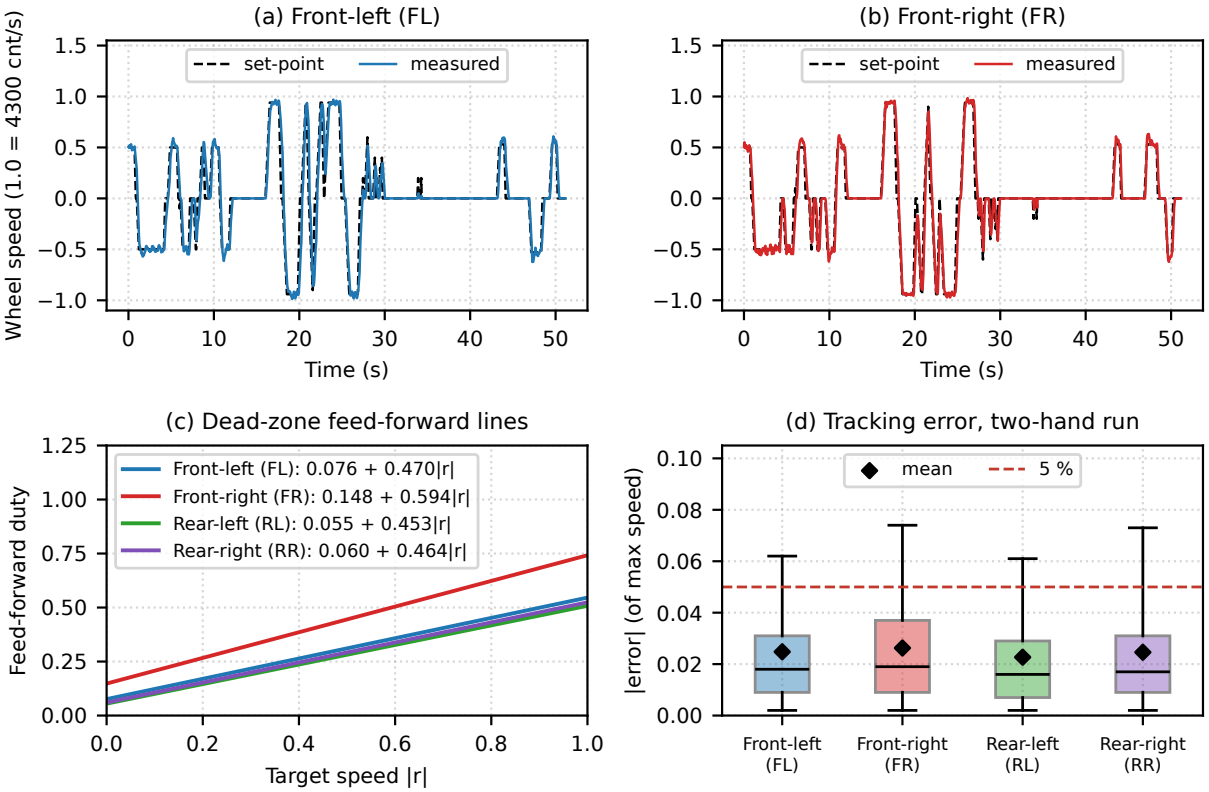


Figure 8: Wheel speed control, two-hand run. (a), (b): set-point and measured speed of two wheels over 50 s; (c): feed-forward lines; (d): distribution of the steady-state error (whiskers: 5th–95th percentile; diamond: mean; dashed line: 5 %).

Body motion. Table 6 compares the command with the velocity computed from the encoders using (2); Fig. 9 shows the RTT distribution and the velocity magnitudes. The deviation on the main axis is below 0.6 % for translation and 2.4–2.5 % for rotation, and unintended motion on the other axes is at most 0.004. These are velocities inferred from the encoders and do not include wheel slip on the floor, so they are not true position errors.

Table 6: Command versus forward kinematics. The first six rows are from the v1 gesture run, the two diagonal rows from the v2 run.

Motion	Samples	Command (v_x, v_y, ω)	From encoders	Main-axis dev.	Other axis
Forward	543	(+0.472; 0; 0)	(+0.474; +0.001; +0.002)	+0.6 %	0.002
Backward	360	(-0.510; 0; 0)	(-0.510; -0.001; -0.001)	-0.1 %	0.001
Left	96	(0; +0.480; 0)	(+0.004; +0.482; +0.002)	+0.4 %	0.004
Right	73	(0; -0.480; 0)	(-0.003; -0.479; 0)	-0.2 %	0.003
Rotate left	238	(0; 0; +0.435)	(+0.003; +0.001; +0.446)	+2.5 %	0.003
Rotate right	230	(0; 0; -0.442)	(-0.001; 0; -0.453)	+2.4 %	0.001
Forward-left (v2)	98	(+0.308; +0.245; 0)	(+0.316; +0.256; +0.001)	–	–
Forward-right (v2)	71	(+0.313; -0.237; 0)	(+0.316; -0.243; -0.002)	–	–

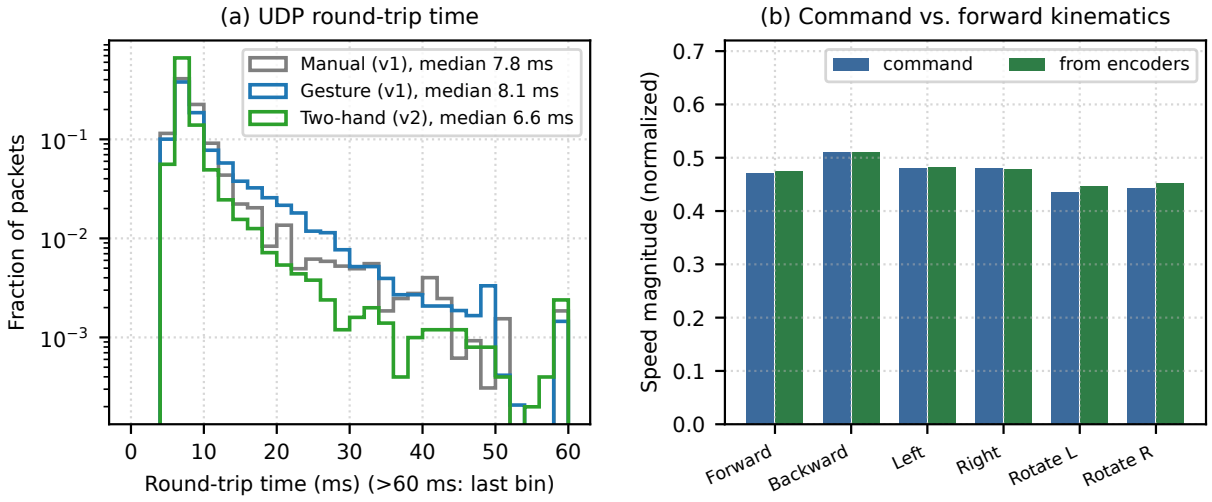


Figure 9: (a) RTT distribution of the three runs (log scale, values above 60 ms fall into the last bin). (b) Magnitude of the commanded velocity and of the velocity from the encoders for the six basic motions.

5.3 Second Session: Vision Loop with Hands, End-to-End Latency and the Cursor near the Frame Edge

After the runs above we recorded four more server sessions with the per-run logger (`web/runlog.py`): one log row per camera frame, plus the 20 Hz drive log when the robot was connected (data in `docs/data/v2/session2/`, table 7). Sessions A and B ran the code from before the changes described below (MJPG 640 × 480 capture, cursor mapped 1:1 to the fingertip); sessions C and D ran after the capture was raised to 1280 × 720 (hand detection still on a 640-px-wide copy) and after the cursor was made to move faster than the fingertip near the frame edge (C with a piecewise-linear mapping, D with the smooth mapping of Section 4.2). A and B had the robot connected; C and D tested the cursor only. A is the first run after a wheel fault was repaired, and D was still running when its log was copied, so D is a snapshot. One operator took part in all four sessions, and the sessions differ in what the operator did, in the number of open browser tabs and in duration, so they are not a controlled comparison.

Table 7: Server sessions recorded with the per-run logger. Mean hand-landmark inference time (ms) is split by the number of hands in the frame; fps is the number of AI frames per second over the whole session.

Sess.	Camera	Cursor	Time (s)	fps	0 hand	1 hand	2 hands
A	640×480	1:1	186	25.5	27.7	45.5	43.3
B	640×480	1:1	76	21.9	31.3	50.1[a]	47.5
C	1280×720	piecewise	363	18.0	30.7	55.5	48.3
D	1280×720	smooth	214	21.9	28.9	52.4	47.2

[a]Only 46 frames with exactly one hand.

Vision loop with hands. With no hand in view, inference averaged 27.7–31.3 ms; with one hand 45.5–55.5 ms and with two hands 43.3–48.3 ms (Fig. 10a). The frame rate over a whole session was 18.0–25.5 fps, below the 25 fps target and below the 29.9 fps of the isolated benchmark above, which measured capture and detection alone with no hand; the server also encodes and streams every frame and runs the control loop, and the cause of the difference was not isolated. Frames with one hand were not cheaper than frames with two (a possible reason, that the detector keeps searching for the missing second hand, was not tested). Raising the capture to

1280 × 720 did not change the no-hand cost (27.7–31.3 ms in all four sessions); with one hand the mean was higher in C and D than in A (55.5 and 52.4 against 45.5 ms), but the sessions are not comparable enough to attribute this to the resolution.

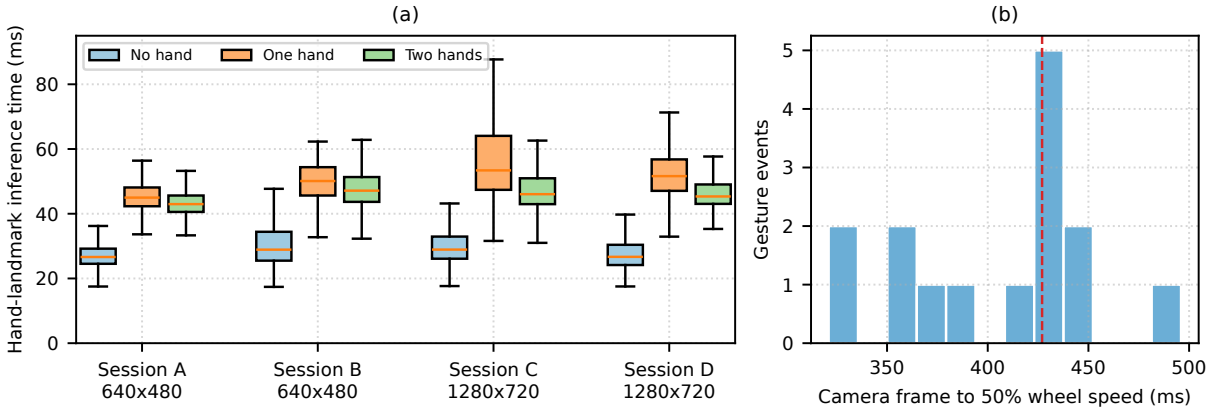


Figure 10: (a) Hand-landmark inference time per session and number of hands in view (boxes: quartiles; whiskers without outliers). (b) Delay from the camera frame in which the gesture filter confirmed a motion gesture to the moment the most loaded wheel reached 50 % of its new target, session A (15 events; dashed: median).

End-to-end latency (session A). For 15 confirmed motion gestures, the median time from the camera frame in which the gesture filter fired to the moment the most loaded wheel reached 50 % of its new target was 427 ms (321–496 ms, Fig. 10b); the command left the laptop 37 ms after that frame (median). The filter itself needs 4 consecutive frames, about three frame intervals (120–140 ms at these frame rates) before the confirming frame, which is not included. The median target magnitude was 0.81 of full speed; with the host acceleration limit of 2.0 s^{-1} half of that takes about 0.2 s, so a large part of the delay is the deliberate ramp. This is longer than the 257–260 ms reported in Section 5.2, which started from the gesture value written in the drive log instead of the camera frame (about 37 ms later) and used targets of other sizes; the 15 events here are too few and too uniform to separate these causes. Session B contains a single event (319 ms, target 0.5).

Wheel tracking and link. With the same rule as in Section 5.2, the mean error was 1.3 % (manual) and 1.5 % (gesture) of full scale in A, and 0.9 % in B (gesture only; 12 manual samples are too few to report), with 95th percentiles of 4.3, 5.0 and 4.2 %. In A the duty was at the 0.95 ceiling in 8.0 % of the manual and 3.8 % of the gesture samples, against 0.0 % in B: at the higher speeds of A the controller ran out of voltage headroom, so the commanded speed could not be reached, which can distort diagonal motion if the wheels saturate unequally (not measured). The UDP round trip had a median of 6.4 and 7.9 ms (95th percentile 23.9 and 17.3 ms; maximum 104 and 47 ms), consistent with the earlier runs.

Cursor near the frame edge. The mapping of Section 4.2 was introduced after the operator reported that the UI hand was lost whenever the fingertip had to reach the frame border to hit the speed slider in the screen corner, and then that the cursor ran ahead of the fingertip when the stretch started abruptly. The logs give an indirect view only, because they do not store the raw fingertip position: the share of UI-hand frames with the cursor within 2 % of a screen edge was 2.3 % in A and 1.4 % in B (1:1 mapping), and 8.2 % in C and 4.6 % in D (stretched); the number of UI-hand segments longer than 0.3 s that ended with the cursor at an edge, a sign of losing the hand there, was 1 of 6 in A, 1 of 3 in B, 0 of 15 in C and 1 of 9 in D. The operator also needed the corner more in C and D because the task was to reach it, so these numbers show that the cursor reaches the screen edge with the stretched mapping, not that hand loss was reduced; the counts are small. The lead of the cursor over the fingertip near the edge (up

to 6 % of the frame width by construction) was not measured against the fingertip position and rests on the operator's report.

6 Limitations and Reproduction

Limitations. The data come from one robot, one floor and one operator. There is no user study. The real-hand cursor measurements (jitter, click shift) come from one person and 12 clicks, and the click hit rate was not measured; the FREEZE_BACK anchor did not give better results in these measurements. There is no comparison of the controller with a plain PID or open loop. The end-to-end latency was measured in one session (15 events), and the effect of the edge mapping on hand loss was not measured against the true fingertip position. The robot has no obstacle sensing.

Reproducing the data and figures. From the project root:

`.venv/bin/python reports/make_figures.py` (generates the figures and `reports/figures/stats.json`)
`python tests/analyze_drive_log.py docs/data/v2/drive_twohand.csv`
`.venv/bin/python reports/session2.py` (second session: `stats_session2.json` and figure)
`python tests/analyze_run.py logs/ai_<time>.csv logs/drive_<time>.csv`
`python tests/test_hands_logic.py` `python tests/camera_fps_test.py`